

Competitive Programming

Section 6: Two pointers and sliding windows

Competitive Programming Course

2026-07-04

Table of contents

1	Why two pointers matter	3
2	Two pointers on a sorted array	4
3	Sliding windows	5
4	Why nonnegative values matter	6
5	Counting many windows at once	7
6	Pointer movement cost	8
7	Worked example: longest bounded window	9
8	Common patterns	10
9	C++ implementation notes	11
10	Python implementation notes	12
11	Proof habits	13
12	Common mistakes	14
13	Before you submit	15
14	Exercises	16

1 Why two pointers matter

Two pointers turn some nested-loop searches into linear scans. The pattern is:

```
move one pointer forward  
adjust the other pointer only forward  
maintain an invariant
```

The important phrase is **only forward**. If each pointer moves at most n times, the total work is $O(n)$, even though the code may contain a nested **while** loop.

Two common forms are:

- two pointers from both ends of a sorted array;
- a sliding window over a contiguous segment.

2 Two pointers on a sorted array

Suppose the array is sorted:

```
1 2 4 7 11
```

and we want to know whether two values sum to 9.

Start with:

```
left = 0  
right = n - 1
```

Trace:

left value	right value	sum	action
1	11	12	too large, move right left
1	7	8	too small, move left right
2	7	9	found

The sorted order tells us which pointer to move. If the sum is too small, the left value is too small to pair with the current right value or any smaller right value. If the sum is too large, the right value is too large to pair with the current left value or any larger left value.

3 Sliding windows

A sliding window represents a contiguous segment:

```
[left, right]
```

For nonnegative arrays, increasing **right** can only increase or preserve the window sum. Increasing **left** can only decrease or preserve it.

That monotonicity is what makes the window work.

For the task:

longest subarray with sum at most **k**

the algorithm is:

```
left = 0
sum = 0
best = 0
for right from 0 to n - 1:
    add a[right]
    while sum > k:
        remove a[left]
        left += 1
    best = max(best, right - left + 1)
```

After the **while** loop, the window is valid. Since all values are nonnegative, moving **left** forward was the only way to make an invalid window valid again.

4 Why nonnegative values matter

Sliding windows over sums usually require nonnegative values.

If values can be negative, removing from the left may increase the sum, and adding to the right may decrease it. The monotonic argument breaks.

Example:

```
10 -100 10
```

The sum does not move predictably as the window expands. For arbitrary signs, prefix sums and maps are often the right tool instead, as in Section 4.

5 Counting many windows at once

For nonnegative arrays, once `[left, right]` is the smallest-left valid window after shrinking, every subarray ending at `right` and starting at any position from `left` through `right` is also valid.

There are:

```
right - left + 1
```

such subarrays.

Example:

```
values = 1 2 3  
k = 3
```

At `right = 1`, the window `[0,1]` has sum 3. Valid subarrays ending at 1 are:

```
[1,2]  
[2]
```

There are 2, which is `right - left + 1`.

6 Pointer movement cost

The code:

```
for right:
    while invalid:
        left += 1
```

looks nested. But `left` never moves backward. Across the whole run, `right` moves `n` times and `left` moves at most `n` times. The total is linear.

This is called amortized analysis:

```
total pointer moves <= 2n
```

7 Worked example: longest bounded window

Array:

2 1 3 4 2

Limit:

7

Trace:

right	add	window after shrinking	sum	best
0	2	[2]	2	1
1	1	[2,1]	3	2
2	3	[2,1,3]	6	3
3	4	[1,3,4] 8 then shrink to [3,4]		7
4	2	[3,4,2] then shrink to [4,2]	6	3

The answer is 3.

8 Common patterns

Two pointers work when there is a monotone reason to move one pointer:

- sorted two-sum;
- longest subarray under a limit with nonnegative values;
- counting subarrays under a limit with nonnegative values;
- removing duplicates in sorted arrays;
- merging sorted lists.

The pattern is suspect when:

- values can be negative and the condition is based on sum;
- both pointers may need to move backward;
- the input is unsorted and no monotone structure exists.

9 C++ implementation notes

Use `long long` for sums and answers. The number of subarrays can be:

$$\frac{n(n+1)}{2}.$$

For `n = 200000`, that is about $2 * 10^{\{10\}}$.

10 Python implementation notes

Python loops are fast enough for linear two-pointer solutions at this scale. Keep input parsing simple and avoid slicing inside the main loop.

11 Proof habits

For sorted two-sum, prove that the discarded value cannot participate in any future solution.

For sliding windows, prove:

1. after shrinking, the current window is valid;
2. every counted subarray is valid;
3. no valid subarray ending at the current right endpoint is missed;
4. each pointer moves only forward.

12 Common mistakes

- Using sliding window with negative values.
- Forgetting to shrink repeatedly with `while`.
- Counting only one subarray ending at each right endpoint.
- Moving both pointers at once in sorted two-sum after a failed comparison.
- Allowing `left == right` to count the same element twice in two-sum.
- Using 32-bit `int` for subarray counts.

13 Before you submit

Ask:

1. What monotonic property lets a pointer move only forward?
2. Are values nonnegative if the condition is about sums?
3. Is the input sorted, or do I need to sort it first?
4. Does the task ask for existence, maximum length, or count?
5. Can the answer exceed 32-bit range?

14 Exercises

The local exercises cover:

1. `a_sorted_two_sum`: two pointers from both ends.
2. `b_longest_bounded_window`: maintain a valid window.
3. `c_count_subarrays_at_most`: count all valid starts for each right endpoint.

Run:

```
python3 sections/06_two_pointers_sliding_windows/check.py
```